HABITFLOW

Habit Tracker App

Complete Feature Requirements Document

A production-style Flutter internship assignment covering habit creation, daily tracking, streaks, reminders, history, analytics, local storage, and quality assurance.

Document type	Product feature specification / internship brief
Platform	Android mobile application built with Flutter and Dart
Suggested timeline	7 working days
Architecture	Feature-based, layered, offline-first
Backend	Not required for the core assignment

Product goal
Help users build consistent routines by making habits easy to schedule, complete, review, and improve - while keeping their data reliable and available offline.

Prepared as an implementation-ready brief for a Flutter Developer Intern. P0 requirements are mandatory; P1 and stretch goals should only be attempted after core behavior is correct.

Contents

1. Product overview and scope
2. Feature priorities
3. Screen-by-screen requirements
4. Streak and completion calculation rules
5. UI/UX and design requirements
6. Technical architecture
7. Data model and integrity
8. Functional and non-functional requirements
9. QA test cases
10. Seven-day implementation plan
11. Submission checklist
12. Final acceptance criteria and evaluation rubric

1. Product overview and scope

HabitFlow is a personal habit-building application. Users create habits, configure recurring schedules, record daily progress, maintain streaks, receive optional reminders, and review performance over time.

Target users

- Students building study, reading, or exercise routines.
- Professionals organizing daily productivity habits.
- Users tracking hydration, mindfulness, sleep routines, or personal goals.
- People who want a clear view of consistency through calendars and statistics.

Product principles

- Fast daily check-ins with minimal friction.
- Reliable historical records and understandable streak rules.
- Immediate, accessible feedback after user actions.
- Persistent local data across app restarts.
- A polished interface with empty, loading, error, and confirmation states.

2. Feature priorities

Priority	Features	Definition
P0 - Mandatory	Onboarding, dashboard, habit CRUD, schedules, daily completion, streaks, calendar history, basic statistics, local persistence, settings, core tests	Must work for submission.
P1 - Polish	Local notifications, categories, edit/delete confirmation, dark mode, empty/error states, JSON export	Expected where feasible after P0.
Stretch	Home-screen widgets, achievements, backup/restore, configurable themes	Optional; must not replace core quality.

Scope boundary: No login, backend, Firebase, or paid API is required. The application must be useful offline.

3. Screen-by-screen requirements

3.1 Onboarding

- Show app name, welcome illustration, and a concise explanation of habit tracking.
- Provide a Get Started action and optional preferred category selection.
- Persist onboarding completion locally; do not show onboarding on every launch.
- Request notification permission in context. The app must remain usable if permission is denied.

3.2 Home dashboard

- Show greeting, current local date, daily progress ring or bar, and completed-versus-scheduled counts.
- List habits scheduled for the selected date with clear completed and pending states.
- Provide quick completion actions, add-habit action, and a streak summary.
- Provide navigation to Today, History, Statistics, and Settings.
- Show a helpful empty state and create-first-habit action when nothing is scheduled.
- Refresh after returning from detail screens and recalculate correctly when the date changes.

Dashboard calculation example: if 3 of 5 habits scheduled today are complete, display 3/5 and 60%. Do not divide by every habit ever created.

3.3 Create and edit habit

- Required name; optional description; category; icon; color; tracking type; target; schedule; optional reminder.
- Schedule options: every day or selected weekdays. Custom weekly goals may be an extension.
- Trim whitespace; reject an empty name; use a sensible maximum name length (e.g. 60 characters).
- Numeric and duration targets must be greater than zero. Weekday schedules must select at least one day.
- Save only after validation. A saved habit must appear on its applicable dates.
- Editing a schedule must not erase historical completion records.

Tracking types

Type	Example	Completion rule
Binary	Read daily	User marks done or not done.
Numeric	Drink 2,000 ml water	Progress reaches configured numeric target.
Duration	Meditate 20 minutes	Recorded minutes reach duration target.

3.4 Habit details and management

- Show habit name, category, tracking type, target, schedule, reminder, current streak, best streak, and recent history.
- Allow edit, archive, and delete actions.
- Ask for confirmation before deletion and clean up associated reminders.
- Prefer archive when users want to stop tracking but keep history.

3.5 Daily tracking

- Binary: tap to complete; tap again to undo. Do not create duplicate records for the same habit/date.
- Numeric: add or correct progress; show value/target and percentage; mark complete at target.
- Duration: record completed minutes; a running timer is optional.

3. Screen-by-screen requirements (continued)

3.6 History and calendar

- Display a monthly calendar with markers for completed, missed scheduled, and unscheduled dates.
- Tap a date to inspect habits and progress for that day.
- Support previous/next month navigation and habit-specific history.
- Do not mark future dates as missed. Preserve history for archived habits.
- Show a clear empty state when no habits were scheduled.

3.7 Statistics and analytics

- Today's, weekly, and monthly completion rates.
- Current streak, best streak, total completed occurrences, and habit-level performance.
- Weekly or monthly trend chart derived from persisted records.
- Exclude dates before a habit's start date, future occurrences, and unscheduled dates.
- Numeric and duration habits count as complete only when their targets are met.
- If no eligible occurrences exist, show 'No scheduled habits' rather than NaN or an invalid percentage.

Completion rate = completed eligible occurrences / total eligible occurrences x 100.

3.8 Local reminders

- Allow reminder enable/disable and local time selection per habit.
- Schedule only on applicable weekdays.
- Update pending notifications after a schedule or time change.
- Cancel notifications when reminders are disabled or a habit is deleted.
- Handle denied permission and platform-specific notification limits.
- Avoid duplicate reminders after repeated edits; reschedule after relevant device/app events when required by the chosen plugin.

3.9 Categories and personalization

- Predefined categories: Health, Fitness, Learning, Productivity, Mindfulness, Personal, and Other.
- Allow icon and color selection for each habit.
- Support category filters on dashboard, history, and statistics where appropriate.
- Personalization must not change tracking logic.

3.10 Settings

- Light, dark, and system theme; week-start preference; notification status.
- Optional default reminder preference; export data; reset application data; About/version.
- Reset requires confirmation and clearly explains that local habits, history, and preferences will be removed.
- Clean up reminders when resetting data.

3.11 Data export/import (P1)

- Export habits, schedules, completion history, and supported preferences to JSON.
- Validate import data before modifying stored records.

- Malformed imports must not corrupt existing data.
- Document whether import merges or replaces existing data and prevent accidental duplicate records.
- If time is limited, validated JSON export is sufficient for the internship version.

4. Streak and completion calculation rules

Implement the streak engine as a separate, testable Dart service rather than embedding calculations in widgets.

- **Current streak:** consecutive scheduled habit dates completed according to the documented policy.
- **Best streak:** longest historical sequence of qualifying completed scheduled dates.
- **Missed occurrence:** a scheduled date that has passed without satisfying the target.
- Only scheduled dates count; unscheduled days do not break a streak.
- The current day must not be counted as missed before the day ends.
- Editing a future schedule must not rewrite historical completion records.
- Handle local dates and time-zone/daylight-saving changes consistently.

Example: a habit scheduled Monday, Wednesday, and Friday. Completing Monday and Wednesday gives a streak of 2; Tuesday and Thursday do not affect it. If Friday ends incomplete, the current streak is interrupted. The next completed scheduled occurrence starts a new streak.

The intern must document how a habit scheduled today but not yet completed is treated in the current-streak calculation.

5. UI/UX and design requirements

Token	Value	Usage
Primary green	#16A34A	Primary actions, completed states, progress.
Accent green	#22C55E	Highlights and progress accents.
Light background	#F0FDF4	Soft light-theme surfaces.
Dark background	#111827	Dark-theme background.

- Use Poppins or Inter consistently.
- Keep spacing, border radii, icon sizes, and button styles consistent.
- Support common Android screen sizes; respect system insets and keyboard.
- Provide accessible labels, readable contrast, and usable touch targets.
- Avoid overflow on small screens and ensure both themes remain readable.
- Use subtle completion animations without delaying interaction.
- Provide loading, empty, error, success, and confirmation states.

Recommended navigation

- Today: dashboard and daily tracking.
- History: calendar and past completion.
- Statistics: trends and performance.
- Settings: preferences and data management.
- Habit creation and details can be separate routes.

6. Technical architecture

Area	Recommendation
Framework	Flutter / Dart
State management	Riverpod or Bloc; justify the choice.
Navigation	go_router or Flutter Navigator
Local database	SQLite with Drift or another justified persistent database
Notifications	A maintained Flutter local-notification package
Charts	A maintained Flutter chart package
Testing	flutter_test and mocks/fakes where useful
Version control	Git with meaningful commits

Suggested folder structure

lib/
app/ (app.dart, router.dart, theme/)
core/ (database/, notifications/, utils/, widgets/)
features/
onboarding/ (data/, domain/, presentation/)
habits/ (data/, domain/, presentation/)
tracking/ (data/, domain/, presentation/)
history/ (data/, domain/, presentation/)
statistics/ (domain/, presentation/)
settings/ (data/, presentation/)
main.dart
test/ (unit/, widget/, integration/)

Layer responsibilities

- **Presentation:** screens, reusable widgets, UI state.
- **Domain:** habit rules, completion calculations, streak logic.
- **Data:** models, database operations, repositories.
- **Services:** notifications, date utilities, export/import.
- **Tests:** business rules independent of UI.

Widgets should not contain direct database queries or duplicated streak-calculation logic.

7. Data model and integrity

Habit entity

Field	Purpose
id	Stable unique identifier
name / description	Habit label and optional details
category / icon / color	Organization and appearance
trackingType	Binary, numeric, or duration
targetValue / unit	Target amount and unit, where applicable
scheduleType / scheduledWeekdays	Daily or selected weekdays
startDate	First eligible local date
reminderTime	Optional local reminder time
isArchived	Whether the habit is active
createdAt / updatedAt	Creation and modification timestamps

Habit completion entity

Field	Purpose
id	Unique record identifier
habitId	Associated habit identifier
localDate	Calendar date of the record
progressValue	Recorded amount
isCompleted	Whether the target was satisfied
completedAt	Optional completion timestamp

Data integrity rules

- Every completion references an existing habit.
- Habit IDs remain stable when names or schedules change.
- Dates are stored and compared consistently.
- Updates do not create duplicate completion records.
- Ordinary edits preserve historical data.
- Database failures produce recoverable errors rather than silent data loss.
- Document deletion, archival, and import behavior.

Use a unique constraint on (habitId, localDate) for one aggregate daily record. If multiple numeric entries per day are supported, store individual entries separately and calculate their aggregate.

8. Functional and non-functional requirements

Functional requirements

ID	Requirement
FR-01	Create, edit, archive, and delete habits.
FR-02	Configure daily and weekday-based schedules.
FR-03	Track binary, numeric, and duration progress.
FR-04	Calculate streaks using scheduled occurrences.
FR-05	Display date-specific completion history.
FR-06	Calculate statistics from persisted records.
FR-07	Schedule and manage local reminders.
FR-08	Save preferences and restore them after restart.
FR-09	Support theme selection and category filtering.
FR-10	Validate input and handle expected errors.

Non-functional requirements

- **Reliability:** data survives force-close and normal device restart.
- **Performance:** common actions feel immediate; avoid unnecessary full-screen refreshes.
- **Maintainability:** business rules are independently testable.
- **Accessibility:** controls have meaningful labels and usable touch targets.
- **Compatibility:** test supported Android versions/devices for the selected Flutter SDK and plugins.
- **Privacy:** data stays on-device unless the user explicitly exports it.
- **Resilience:** invalid input, denied permissions, and database errors do not crash the app.

9. QA test cases

ID	Scenario	Expected result
QA-01	Create a valid habit	Appears on applicable scheduled dates.
QA-02	Empty habit name	Validation error; no record created.
QA-03	Complete binary habit twice	Second action undoes completion; no duplicate.
QA-04	Record partial numeric progress	Progress persists; incomplete below target.
QA-05	Reach numeric target	Completion and statistics update.
QA-06	Restart application	Habits, progress, theme, and schedules persist.
QA-07	Weekday-only habit	Unscheduled days do not break streak.
QA-08	Miss a scheduled date	Streak follows documented date policy.
QA-09	Edit schedule	Future eligibility changes; history remains intact.
QA-10	Deny notification permission	App remains usable; limitations explained.
QA-11	Archive a habit	Removed from active lists; history remains.
QA-12	Delete a habit	Confirmation and documented cleanup occur.
QA-13	No eligible habits on date	No invalid percentage or false missed state.
QA-14	Switch themes	Screens readable; preference persists.
QA-15	Import malformed JSON	Fails safely without corrupting data.
QA-16	Small screen	No overflow or inaccessible actions.

Automated testing minimum

- Unit tests for streaks, completion rates, schedule eligibility, and date boundaries.
- Tests for numeric targets and progress updates.
- Repository/database tests for create, update, archive, and delete.
- Widget tests for creating a habit and marking it complete.
- Tests for empty/error states and import validation.
- Manual regression checklist with actual pass/fail evidence.

10. Seven-day implementation plan

Day	Tasks	Deliverable
1	Review requirements; plan UI, architecture, and schema.	Project structure and navigation.
2	Implement create/edit/delete and schedules.	Working habit management.
3	Build dashboard and daily tracking.	Persistent daily progress.
4	Implement streak engine and calendar.	Correct historical tracking.
5	Add statistics, themes, and notifications.	Functional secondary features.
6	Write tests, fix bugs, run regression.	Test evidence and regression results.
7	Final QA, README, screenshots, and APK.	Complete submission.

Prioritization guidance

If time is limited, prioritize correct tracking, persistence, and streak calculations over advanced animations or optional features.

11. Submission checklist

- Git repository with source code and meaningful commit history.
- Working Android debug APK.
- README with setup and build instructions.
- Architecture explanation and database schema.
- Screenshots of major user flows.
- Unit and widget test files with execution results.
- Manual QA checklist with pass/fail results.
- Known issues and limitations.
- Short demonstration video of the main flows.
- Explanation of technical decisions and possible improvements.

12. Final acceptance criteria

- App builds and launches on a supported Android device.
- Users can create, edit, complete, archive, and delete habits.
- Scheduled habits appear on correct dates.
- Progress persists after restart.
- Streaks and statistics match stored records.
- Historical data is not silently lost when a habit is edited.
- Reminders work where permission and platform support allow.
- Core screens handle empty, error, and confirmation states.
- Required automated tests pass.
- A second developer can run the project using the README.

Evaluation rubric

Area	Weight
Functional correctness	25%
Flutter and Dart fundamentals	15%
Architecture and maintainability	15%
UI/UX quality	10%
Data persistence and integrity	10%
Testing and debugging	15%
Git, documentation, and communication	10%
Total	100%

Suggested hiring threshold: 75/100, with no major failure in core data integrity or functional correctness. Treat the score as a decision aid; for an intern, learning speed, debugging approach, and ability to explain trade-offs also matter.

Final live-review challenge

After submission, ask the intern to fix a streak-calculation bug or add a weekly schedule option. Observe whether they investigate the issue, change the correct layer, add a test, and verify existing behavior still works.

End of document - HabitFlow Flutter Internship Feature Requirements